

Topic and Sentiment Evolution in a Politician's Policy Commentary

A defensible tidytext + topic-modeling + sentiment workflow under a Total Survey Error frame

Kevin

2026-06-16

Table of contents

1	Purpose and analytic framing	2
2	Reproducibility: package installation	3
3	Simulation design (the data-generating process)	3
3.1	Topic vocabularies and sentence templates	4
3.2	Time-varying topic prevalence (ground truth)	5
3.3	Time-varying sentiment / valence (ground truth)	6
3.4	Generating the population corpus	8
3.5	Imposing the selection mechanism (turning a census into a sample)	9
4	Text preprocessing with tidytext	10
5	Exploratory text analysis: distinctive terms over time	11
6	Topic model 1: Latent Dirichlet Allocation (book Ch. 6)	12
6.1	Choosing the number of topics	12
6.2	Fitting LDA and extracting topics	13
6.3	Aligning estimated topics to ground truth (Hungarian algorithm)	14
6.4	LDA-based prevalence over time and validation	15
7	Topic model 2: Structural Topic Model (time-aware, recommended)	16
8	Sentiment analysis (book Ch. 2 and Ch. 4)	19
8.1	Sentiment by topic over time, validated against truth	21
8.2	Negation diagnostic (book Ch. 4): how much does unigram sentiment misread? . . .	22
9	Optional: AI-assisted module with a local model (Gemma via Ollama)	23
9.1	Use 1 - LLM topic labeling	24
9.2	Use 2 - LLM sentiment, cross-checked against the lexicon	24

10 Inference under sampling: correcting selection bias	26
10.1 The estimator	26
10.2 Prevalence: naive vs. corrected	27
10.3 Sentiment: naive vs. corrected	28
10.4 Sensitivity analysis: how much does the assumption matter?	30
11 Adapting this workflow to the real corpus	31
12 Limitations and threats to validity	32
13 References	32

1 Purpose and analytic framing

This document specifies a reproducible pipeline for studying **how the policy topics and the sentiment in a politician’s public commentary change over time**. The text-mining methods follow Silge & Robinson, *Text Mining with R: A Tidy Approach* (the tidytext book); the topic and sentiment estimates are then wrapped in a survey-inference frame so the “we did not collect everything” reality is handled explicitly rather than ignored.

The corpus is treated not as a complete record but as a **sample from an unobserved target population** of all posts the politician ever made. That reframing is the core of the workflow’s defensibility and maps the problem onto the **Total Survey Error (TSE)** framework used for survey data:

- **Coverage / frame error.** Which posts could *ever* enter the corpus? Deleted, edited, platform-gap, and private posts are outside the frame; their count is unknown.
- **Selection / “nonresponse” error.** Among posts that exist, only a subset is observed. If capture depends on *when* a post was made (older posts churn off platforms) or on *what it is about* (controversial posts get deleted), naive prevalence and sentiment estimates are biased.
- **Measurement error.** A topic model and a sentiment score are error-prone *measurements*. Lexicon sentiment in particular is unigram-based and misreads negation (“not good”); the book is explicit about this, and we operationalize the caution as a diagnostic. Vocabulary and framing also drift over a decade, threatening measurement comparability (the textual analogue of measurement non-invariance).
- **Processing error.** Tokenization, stop-word lists, stemming, and chunk size are analyst choices that propagate downstream.
- **Adjustment / estimation.** Weighting corrects selection bias *if* the selection mechanism is modeled correctly.

The plan: (1) simulate a population corpus with a **known** time-varying topic structure *and* a **known** sentiment (valence) structure; (2) impose a **known** content-dependent selection mechanism to produce the observed sample; (3) run the tidytext pipeline (frequencies, tf-idf, topics, lexicon sentiment, negation diagnostic); (4) recover topic and sentiment trajectories and **validate them against ground truth**; (5) add an optional AI module using a **local** model (Gemma via Ollama through ellmer) for topic labeling and sentiment, cross-checked against the lexicon; (6) show how design weighting corrects selection bias for both prevalence and sentiment, and how sensitive that

correction is. Because the data-generating process is known, the workflow can demonstrate that it recovers truth before it is pointed at real data.

i Why simulate first

On real found-text data the ground truth is unknown, so you cannot tell whether the pipeline is recovering structure or inventing it. Validating against a known data-generating process is the standard defense. Section 10 lists exactly which lines to swap for the real corpus.

2 Reproducibility: package installation

```
# Run once. {stm} pulls in {tm} and {SnowballC}.
install.packages(c(
  "tidyverse", "tidytext", "topicmodels", "stm", "tm", "SnowballC",
  "srvyr", "survey", "clue", "glue", "scales"
))

# Sentiment lexicons: "bing" ships with tidytext; "afinn"/"nrc" require
# ↪ {textdata}
# and a ONE-TIME interactive license acceptance. Run these once interactively:
install.packages("textdata")
# textdata::lexicon_afinn() # accept license at the prompt
# textdata::lexicon_nrc()  # accept license at the prompt

# Optional AI module (Section 9): local LLMs via Ollama + ellmer.
install.packages(c("ellmer", "irr"))
# - Install Ollama (https://ollama.com), then pull a local model, e.g.:
#   ollama pull gemma3 # or your local gemma/llama tag
# - Confirm the exact local tag from R with ellmer::models_ollama().
```

3 Simulation design (the data-generating process)

All structural parameters live in one place so the simulation is fully specified and tunable.

```
dgp <- list(
  years      = 2014:2025, # decade of commentary
  posts_per_year = 150L,  # "hundreds per year"; raise for realism
  K          = 6L,       # number of latent policy topics
  theta_noise_sd = 0.55,  # document-level dispersion of topic mix
  n_sent_range = 3:6      # sentences per post (paragraph length)
)

topic_names <- c(
```

```

"Economy & Taxation",
"Healthcare",
"Immigration & Borders",
"Climate & Energy",
"National Security & Defense",
"Education"
)

```

3.1 Topic vocabularies and sentence templates

Each latent topic is a small bank of distinctive content words; overlap between banks is minimized so recovery is unambiguous (on real data, topics overlap and recovery is harder, a limitation noted in Section 11). Sentences are assembled from templates carrying realistic function-word noise so the downstream stop-word removal has something to do.

```

topic_vocab <- list(
  c("economy","taxes","taxation","budget","deficit","inflation","wages",
    "growth","tariffs","investment","fiscal","revenue","spending","debt"),
  c("healthcare","hospitals","insurance","patients","doctors","nurses",
    "coverage","clinics","premiums","prescriptions","treatment","medicine"),
  c("immigration","border","migrants","asylum","visas","citizenship",
    "refugees","deportation","integration","quotas","residency","newcomers"),
  c("climate","emissions","renewables","solar","wind","carbon","pollution",
    "sustainability","fossil","electricity","grid","greenhouse"),
  c("defense","military","terrorism","intelligence","cyber","alliances",
    "sovereignty","troops","surveillance","deterrence","drones","missiles"),
  c("education","schools","students","teachers","universities","curriculum",
    "tuition","literacy","classrooms","scholarships","graduates","learning")
)
names(topic_vocab) <- topic_names

# Neutral templates (function-word noise + two topic content words).
templates <- c(
  "We must address {w1} and {w2} for families across the country.",
  "My position on {w1} is clear: the government should act on {w2}.",
  "Today I spoke in Parliament about {w1} and our {w2} policy.",
  "For too long we have debated {w1} and the question of {w2}.",
  "Our plan this session covers {w1} alongside {w2}.",
  "Citizens have raised concerns about {w1} and {w2}.",
  "This budget cycle we will turn to {w1} and {w2}.",
  "The national conversation on {w1} and {w2} continues."
)

```

3.2 Time-varying topic prevalence (ground truth)

Each topic gets an unnormalized intensity as a smooth function of the year index $t = 0, 1, \dots$, mapped to a prevalence vector $\pi(t)$ via a softmax:

$$\pi_k(t) = \frac{\exp\{\eta_k(t)\}}{\sum_{j=1}^K \exp\{\eta_j(t)\}}.$$

Planted trajectories: **Economy** declining, **Healthcare** and **Climate** rising, **Immigration** spiking mid-period, **Security** with a later bump, **Education** roughly flat. These are the curves the pipeline must recover.

```
t_idx <- seq_along(dgp$years) - 1L # 0..(T-1)

eta <- tibble(
  t      = t_idx,
  economy = 2.0 - 0.16 * t,
  health  = 0.4 + 0.20 * t,
  immig   = 2.3 * exp(-0.5 * ((t - 3) / 1.6)^2),
  climate = 0.1 + 0.24 * t,
  security = 0.6 + 1.1 * exp(-0.5 * ((t - 7) / 1.3)^2),
  educ    = 0.9 - 0.02 * t
)

softmax_rows <- function(m) {
  ex <- exp(m - apply(m, 1, max))
  ex / rowSums(ex)
}

pi_mat <- eta |>
  dplyr::select(-t) |>
  as.matrix() |>
  softmax_rows()
colnames(pi_mat) <- topic_names
rownames(pi_mat) <- dgp$years

truth_prevalence <- pi_mat |>
  as_tibble(rownames = "year") |>
  mutate(year = as.integer(year)) |>
  pivot_longer(-year, names_to = "topic", values_to = "true_prev")

truth_prevalence |>
  mutate(topic = factor(topic, levels = topic_names)) |>
  ggplot(aes(year, true_prev, color = topic)) +
  geom_line(linewidth = 0.9) +
  geom_point(size = 1.2) +
  scale_color_viridis_d(end = 0.92) +
```

```

scale_x_continuous(breaks = dgp$years) +
scale_y_continuous(labels = label_percent()) +
labs(x = NULL, y = "Share of attention", color = NULL,
      title = "Ground-truth topic prevalence") +
theme(legend.position = "bottom",
      axis.text.x = element_text(angle = 45, hjust = 1))

```

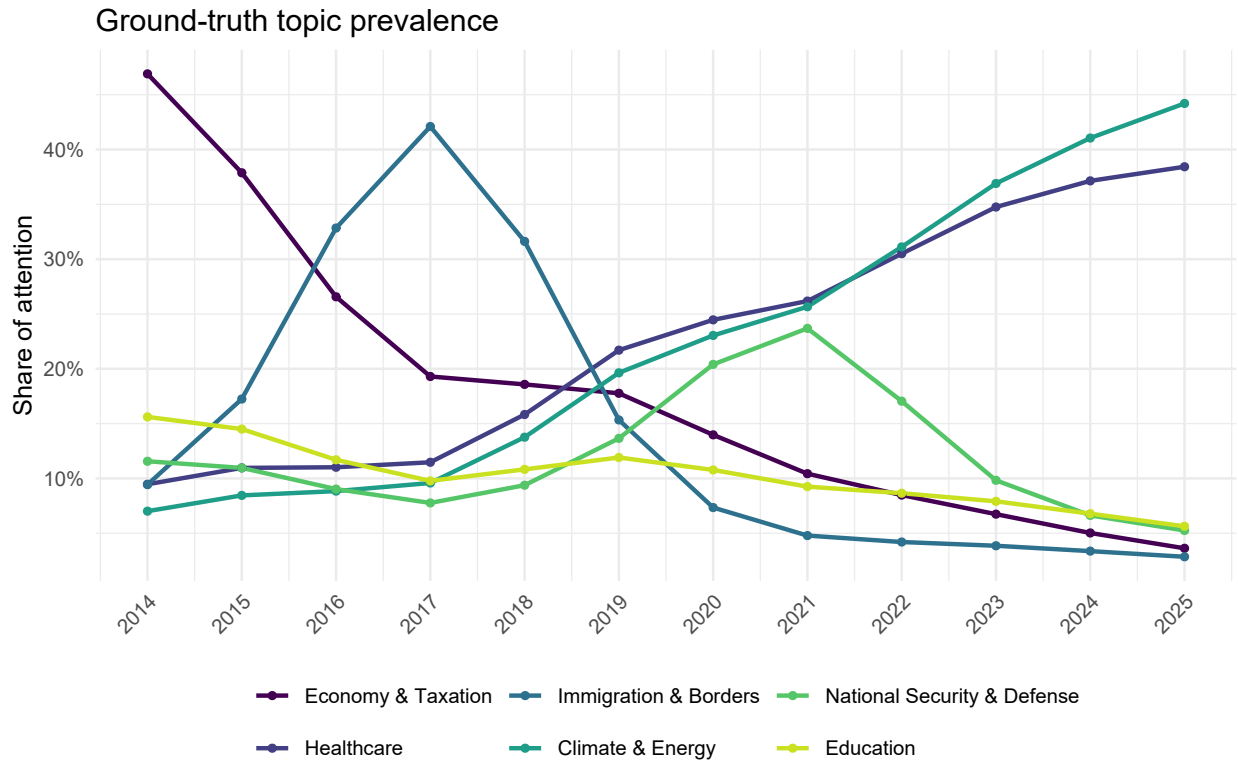


Figure 1: Planted (ground-truth) topic prevalence by year.

3.3 Time-varying sentiment / valence (ground truth)

Sentiment is planted separately from topic. Each topic has a baseline valence, and overall tone **sours over time** (a plausible, recoverable trend). For a sentence on topic k in year-index t , the latent valence is $m_k(t) = v_k + \delta(t - \bar{t})$, and the sentence is drawn positive / neutral / negative with probabilities from a softmax in m . Positive sentences append words from a positive bank, negative sentences from a negative bank; both banks are chosen to exist in the Bing and AFINN lexicons so the measurement can recover them.

```

# Valence words chosen to be present in the Bing lexicon (and mostly AFINN).
valence_words <- list(
  positive = c("support", "benefit", "strong", "proud", "secure", "fair",
               "success", "gain"),

```

```

    negative = c("fail","failed","fear","weak","corrupt","threat",
                "attack","crisis")
)

# Baseline valence per topic (order matches topic_names).
topic_valence <- c(-0.20, 0.10, -0.50, 0.30, -0.60, 0.40)
val_trend_slope <- -0.10 # tone sours ~0.10 logits per year (centered)

sentence_valence_prob <- function(k, t) {
  m <- topic_valence[k] + val_trend_slope * (t - mean(t_idx))
  a <- 1.2
  raw <- c(negative = exp(-a * m), neutral = exp(0.2), positive = exp(a * m))
  raw / sum(raw)
}

make_sentence <- function(k, t) {
  ws <- sample(topic_vocab[[k]], 2)
  base <- glue(sample(templates, 1), w1 = ws[1], w2 = ws[2])
  vp <- sentence_valence_prob(k, t)
  val <- sample(c("negative", "neutral", "positive"), 1, prob = vp)
  if (val == "neutral") return(as.character(base))
  vw <- sample(valence_words[[val]], sample(1:2, 1))
  paste(base, paste(vw, collapse = " "))
}

# Planted net valence (P(positive) - P(negative)) by topic and year = ground
↪ truth.
valence_truth <- expand_grid(topic_i = seq_len(dgp$K), t = t_idx) |>
  mutate(
    probs = map2(topic_i, t, sentence_valence_prob),
    net = map_dbl(probs, \(p) p[["positive"]] - p[["negative"]]),
    topic = topic_names[topic_i],
    year = t + min(dgp$years)
  )

```

```

valence_truth |>
  mutate(topic = factor(topic, levels = topic_names)) |>
  ggplot(aes(year, net, color = topic)) +
  geom_line(linewidth = 0.9) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  geom_hline(yintercept = 0, linewidth = 0.3, linetype = "dotted") +
  labs(x = NULL, y = "Net valence (truth)", color = NULL,
       title = "Ground-truth sentiment by topic over time") +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 45, hjust = 1))

```

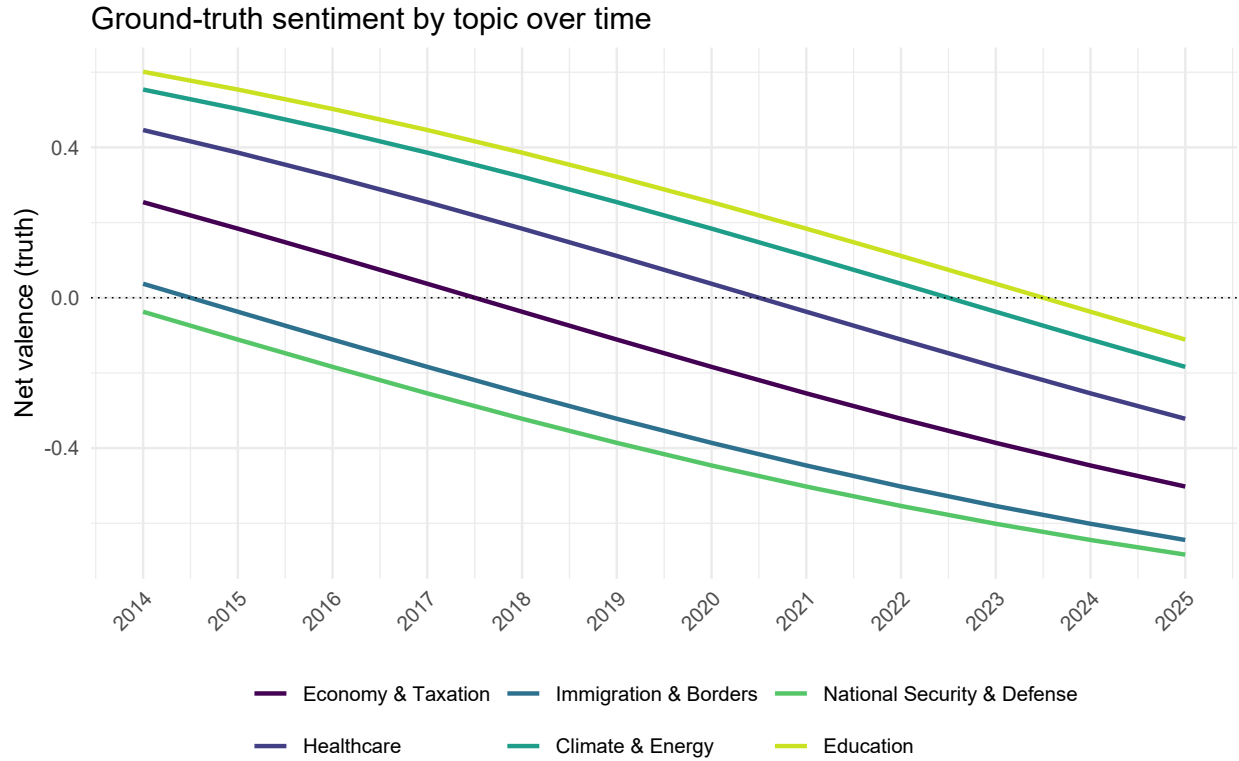


Figure 2: Planted net valence ($P(\text{positive}) - P(\text{negative})$) by topic and year.

3.4 Generating the population corpus

Each post's topic mixture θ_d perturbs the year's prevalence vector on the log scale; each sentence draws a topic from θ_d and a valence from the topic-and-year valence probabilities. Construction is vectorized with `purrr` (no `for/while` loops).

```
draw_theta <- function(t) {
  p <- pi_mat[t + 1L, ]
  raw <- p * exp(rnorm(dgp$K, 0, dgp$theta_noise_sd))
  raw / sum(raw)
}

generate_post <- function(theta, t) {
  n_sent <- sample(dgp$n_sent_range, 1)
  topic_seq <- sample(seq_len(dgp$K), n_sent, replace = TRUE, prob = theta)
  map_chr(topic_seq, \(k) make_sentence(k, t)) |> paste(collapse = " ")
}

set.seed(613)
corpus_pop <- map_dfr(
  dgp$years,
  \(yr) tibble(year = yr, t = yr - min(dgp$years))
```



```

) |>
group_by(year) |>
slice(rep(1, dgp$posts_per_year)) |>
ungroup() |>
mutate(
  doc_id    = sprintf("post_%05d", row_number()),
  theta     = map(t, draw_theta),
  dominant  = map_int(theta, which.max),
  dominant  = factor(topic_names[dominant], levels = topic_names),
  text      = map2_chr(theta, t, generate_post)
)

glue("Population corpus: {nrow(corpus_pop)} posts across ",
     "{min(dgp$years)}-{max(dgp$years)}.")

```

Population corpus: 1800 posts across 2014-2025.

3.5 Imposing the selection mechanism (turning a census into a sample)

The **true** inclusion probability of post d is

$$\pi_d = \text{logit}^{-1}(\beta_0 + \beta_{\text{year}} \tilde{t}_d + \beta_{\text{contro}} c_d),$$

with $\tilde{t}_d$ the centered year (recent posts survive more often) and c_d the post's share of attention on “controversial” topics (Immigration + Security), which the author deletes more often. Because $\beta_{\text{contro}} \neq 0$, capture depends on the *content* being measured: a **missing-not-at-random (MNAR)** mechanism.

```

sel <- list(b0 = 0.4, b_year = 0.16, b_contro = -1.4)
idx_contro <- which(topic_names %in%
                    c("Immigration & Borders", "National Security & Defense"))

corpus_pop <- corpus_pop |>
mutate(
  contro_share = map_dbl(theta, \(th) sum(th[idx_contro])),
  year_c       = t - mean(t),
  pi_true      = plogis(sel$b0 + sel$b_year * year_c +
                        sel$b_contro * contro_share)
)

set.seed(617)
corpus_pop <- corpus_pop |> mutate(observed = rbinom(n(), 1, pi_true) == 1)
observed <- corpus_pop |> filter(observed)

glue("Observed sample: {nrow(observed)} of {nrow(corpus_pop)} posts ",
     "{percent(mean(corpus_pop$observed), 0.1)} capture rate.")

```

Observed sample: 905 of 1800 posts (50.3% capture rate).

```
corpus_pop |>
  group_by(year) |>
  summarise(capture = mean(observed), .groups = "drop") |>
  ggplot(aes(year, capture)) +
  geom_col(fill = viridisLite::viridis(1, begin = 0.35)) +
  scale_x_continuous(breaks = dgp$years) +
  scale_y_continuous(labels = label_percent(), limits = c(0, 1)) +
  labs(x = NULL, y = "Capture rate",
       title = "Sample capture rate by year (selection mechanism)") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

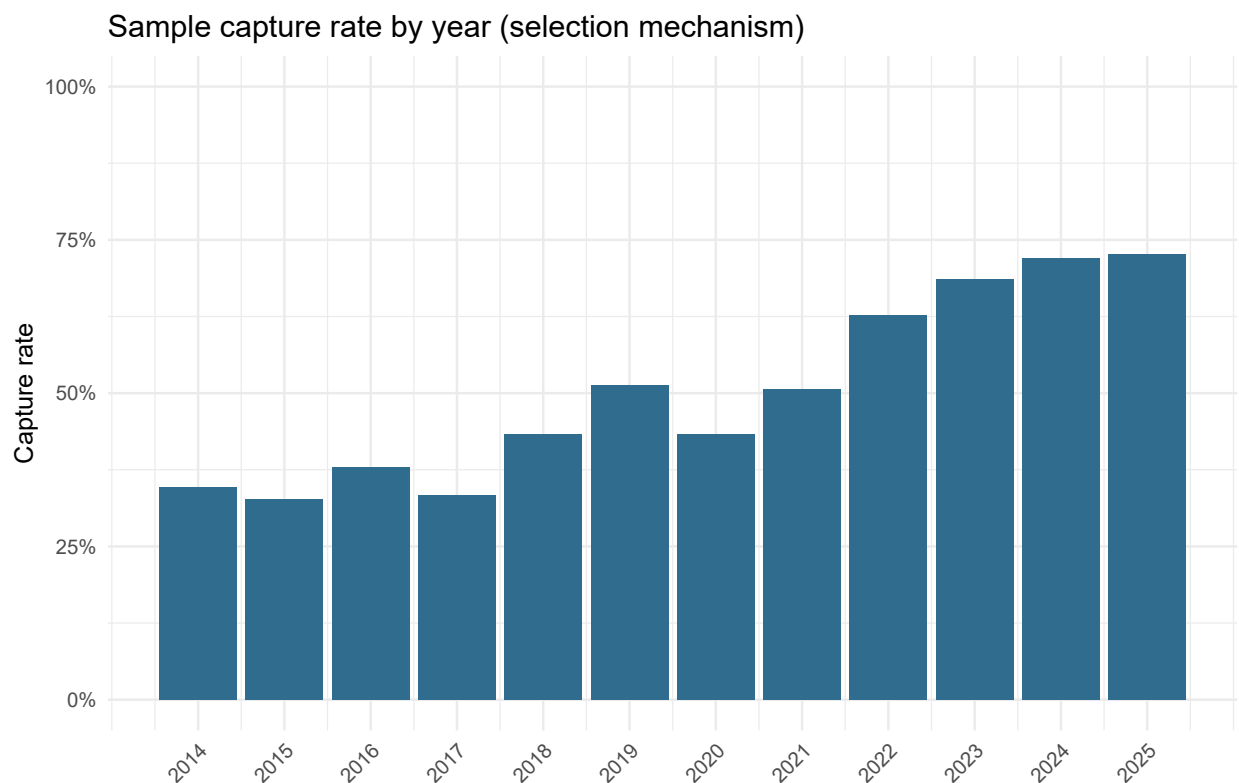


Figure 3: Capture rate by year. Older and controversial posts are under-represented.

4 Text preprocessing with tidytext

From here the code is **identical to what it would be on real data**: it operates on `observed$text`, never on the hidden truth columns. Tokenization uses one-token-per-row form, drops stop words via an anti-join, and removes numeric tokens, following the tidytext book.

```
tokens <- observed |>
  dplyr::select(doc_id, year, text) |>
  unnest_tokens(word, text) |>
  anti_join(get_stopwords(), by = "word") |>
  filter(!str_detect(word, "\\d"))

glue("Vocabulary after cleaning: {n_distinct(tokens$word)} terms; ",
     "{nrow(tokens)} tokens.")
```

Vocabulary after cleaning: 118 terms; 26816 tokens.

Processing-error checkpoint

Stop-word lists, stemming, and minimum-frequency trimming are choices, not neutral defaults. Record them; report a sensitivity check in which headline results are re-estimated under at least one alternative recipe.

5 Exploratory text analysis: distinctive terms over time

Treating each *year* as a document, tf-idf surfaces the terms that distinguish one year's commentary from the decade as a whole (book Ch. 3).

```
tokens |>
  count(year, word) |>
  bind_tf_idf(word, year, n) |>
  group_by(year) |>
  slice_max(tf_idf, n = 6, with_ties = FALSE) |>
  ungroup() |>
  mutate(word = reorder_within(word, tf_idf, year)) |>
  ggplot(aes(tf_idf, word, fill = factor(year))) +
  geom_col(show.legend = FALSE) +
  scale_fill_viridis_d() +
  scale_y_reordered() +
  facet_wrap(~ year, scales = "free_y", ncol = 3) +
  labs(x = "tf-idf", y = NULL, title = "Year-distinctive terms") +
  theme(axis.text.y = element_text(size = 6))
```

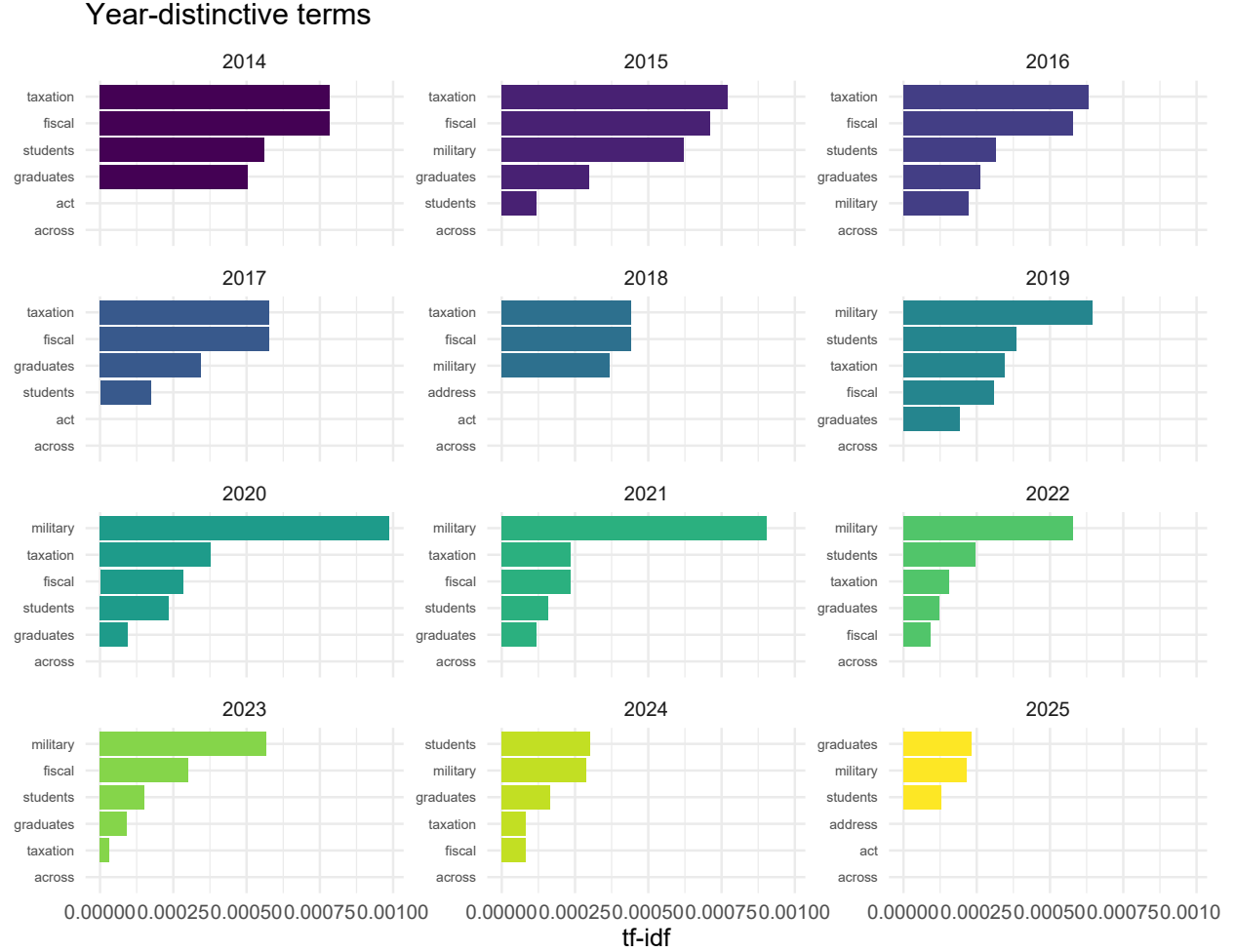


Figure 4: Top tf-idf terms by year (model-free first look at emphasis drift).

6 Topic model 1: Latent Dirichlet Allocation (book Ch. 6)

LDA's generative story is

$$\theta_d \sim \text{Dir}(\alpha), \quad z_{dn} \sim \text{Cat}(\theta_d), \quad w_{dn} \sim \text{Cat}(\beta_{z_{dn}}),$$

with θ_d the document-topic mixture (gamma) and β_k the topic-word distribution (beta). **LDA has no notion of time**; we estimate per-document topic shares and aggregate by year afterward.

6.1 Choosing the number of topics

K is the key tuning parameter. The documented procedure is a held-out perplexity sweep plus interpretability review; it is shown but not executed in the render. We proceed with $K = 6$ because the simulation planted six topics; on real data, **choose K via this procedure, not by assumption.**

```

dtm_all <- observed |> unnest_tokens(word, text) |>
  anti_join(get_stopwords(), by = "word") |>
  filter(!str_detect(word, "\\d")) |>
  count(doc_id, word) |>
  cast_dtm(doc_id, word, n)

map_dfr(4:9, function(k) {
  fit <- LDA(dtm_all, k = k, method = "Gibbs",
             control = list(seed = 1, iter = 800, burnin = 200))
  tibble(k = k, perplexity = perplexity(fit, dtm_all))
}) |>
  ggplot(aes(k, perplexity)) + geom_line() + geom_point() +
  labs(title = "Held-out perplexity by K (lower = better)")

```

6.2 Fitting LDA and extracting topics

```

word_counts <- tokens |> count(doc_id, word)
dtm <- word_counts |> cast_dtm(doc_id, word, n)

lda_fit <- LDA(
  dtm, k = dgp$K, method = "Gibbs",
  control = list(seed = 1234, iter = 1500, burnin = 300, alpha = 0.1)
)

lda_beta <- tidy(lda_fit, matrix = "beta")
lda_gamma <- tidy(lda_fit, matrix = "gamma")

```

```

lda_beta |>
  group_by(topic) |>
  slice_max(beta, n = 8, with_ties = FALSE) |>
  ungroup() |>
  mutate(term = reorder_within(term, beta, topic)) |>
  ggplot(aes(beta, term, fill = factor(topic))) +
  geom_col(show.legend = FALSE) +
  scale_fill_viridis_d() +
  scale_y_reordered() +
  facet_wrap(~ topic, scales = "free_y") +
  labs(x = expression(beta), y = NULL, title = "LDA topic-term distributions")

```

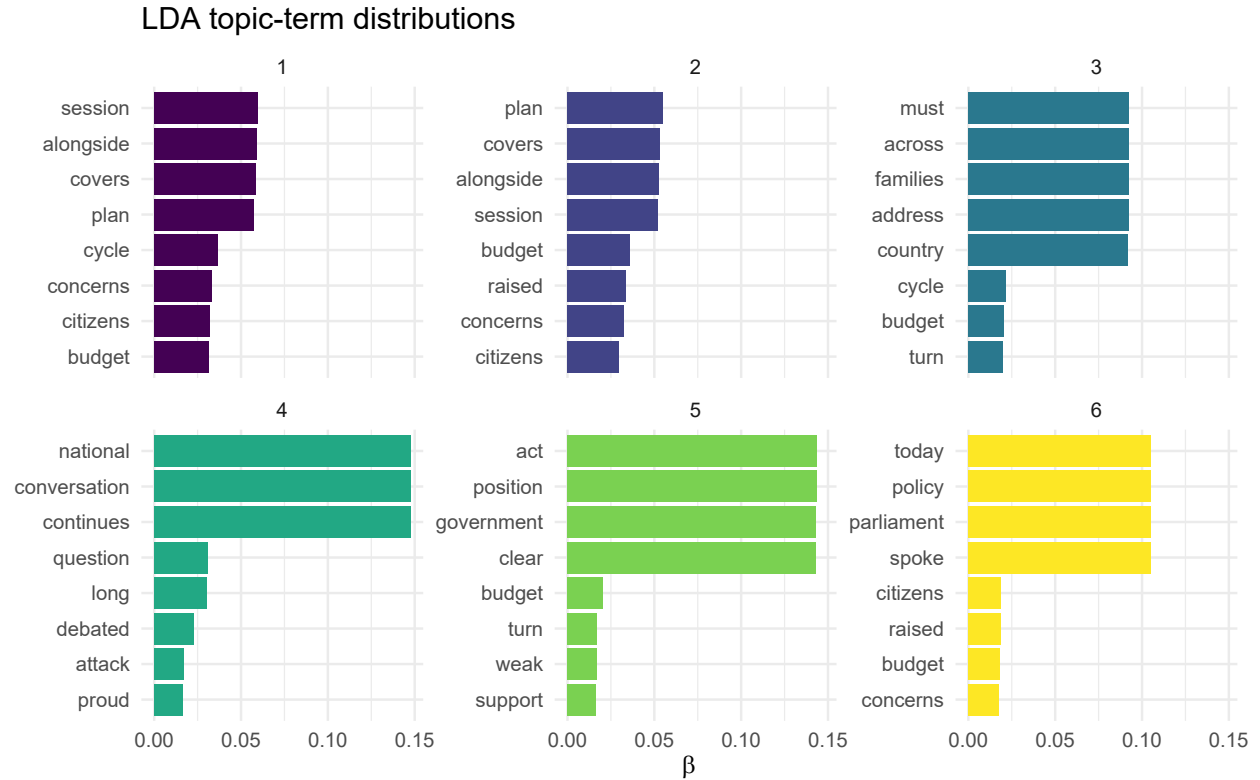


Figure 5: Top terms per LDA topic (numbered arbitrarily until aligned in 7.3).

6.3 Aligning estimated topics to ground truth (Hungarian algorithm)

Topic models return topics in arbitrary order. We align recovered topics to the planted ones by solving a linear assignment problem (Hungarian algorithm) on a term-overlap cost matrix.

```

vocab_lookup <- imap_dfr(topic_vocab, \(words, nm) tibble(term = words, planted =
  ↪ nm))

align_topics <- function(top_terms_df) {
  ov <- top_terms_df |>
    left_join(vocab_lookup, by = "term") |>
    filter(!is.na(planted)) |>
    count(topic, planted) |>
    pivot_wider(names_from = planted, values_from = n, values_fill = 0) |>
    column_to_rownames("planted")
  miss <- setdiff(topic_names, colnames(ov))
  ov[miss] <- 0
  ov <- ov[, topic_names, drop = FALSE]
  a <- solve_LSAP(max(as.matrix(ov)) - as.matrix(ov))
  tibble(topic = as.integer(rownames(ov)),
    planted_name = colnames(ov)[as.integer(a)])
}

```

```
lda_topic_map <- lda_beta |>
  group_by(topic) |>
  slice_max(beta, n = 10, with_ties = FALSE) |>
  ungroup() |>
  rename(term = term) |>
  align_topics()
lda_topic_map
```

```
# A tibble: 5 x 2
  topic planted_name
  <int> <chr>
1     1 Economy & Taxation
2     2 Immigration & Borders
3     3 Climate & Energy
4     5 National Security & Defense
5     6 Education
```

6.4 LDA-based prevalence over time and validation

```
lda_prev <- lda_gamma |>
  rename(doc_id = document) |>
  mutate(topic = as.integer(topic)) |>
  left_join(lda_topic_map, by = "topic") |>
  left_join(dplyr::select(observed, doc_id, year), by = "doc_id") |>
  group_by(year, planted_name) |>
  summarise(est_prev = mean(gamma), .groups = "drop")

lda_prev |>
  ggplot(aes(year, est_prev, color = planted_name)) +
  geom_line(linewidth = 0.9) +
  geom_line(data = truth_prevalence |> rename(planted_name = topic),
            aes(year, true_prev), linetype = "dashed", linewidth = 0.5) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  scale_y_continuous(labels = label_percent()) +
  facet_wrap(~ planted_name) +
  labs(x = NULL, y = "Share of attention",
       title = "LDA recovery vs. ground truth") +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 45, hjust = 1, size = 6))
```

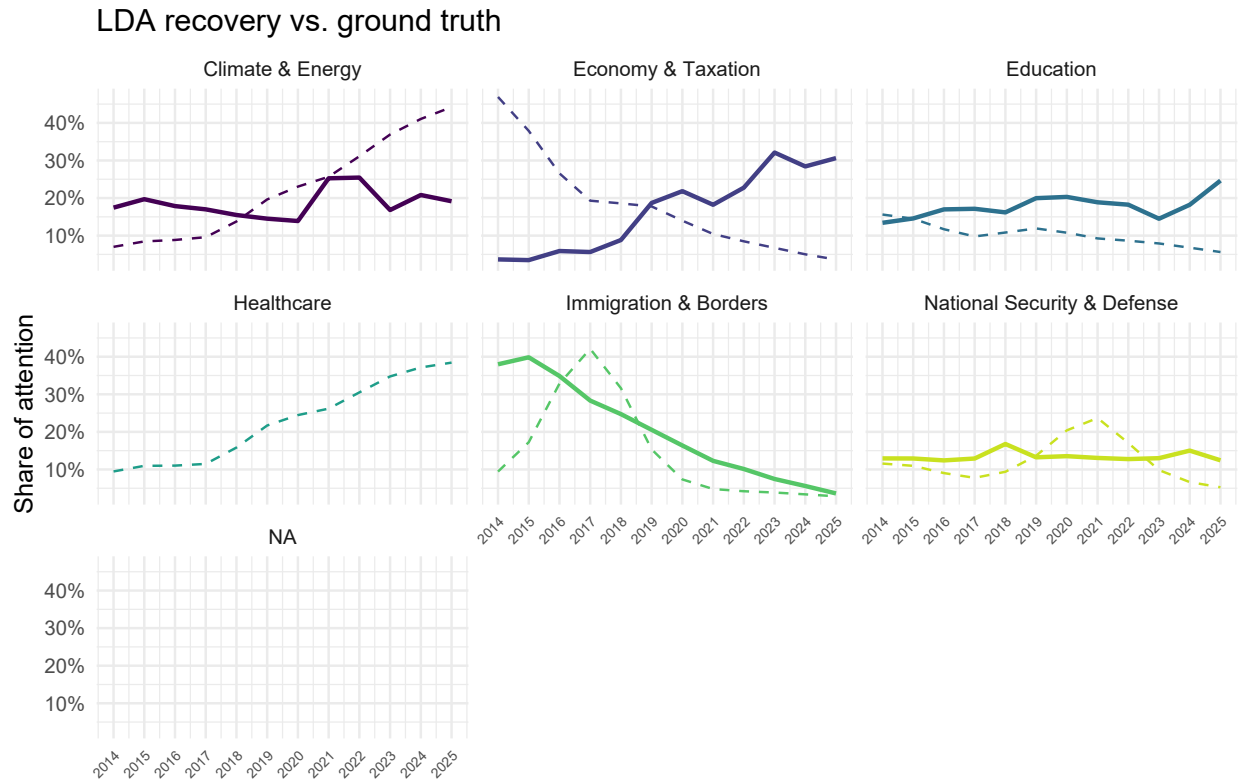


Figure 6: LDA-recovered prevalence (lines) vs. ground truth (dashed).

```
lda_prev |>
  left_join(truth_prevalence, by = c("year", "planted_name" = "topic")) |>
  group_by(planted_name) |>
  summarise(correlation = cor(est_prev, true_prev), .groups = "drop")
```

```
# A tibble: 6 x 2
  planted_name      correlation
  <chr>            <dbl>
1 Climate & Energy 0.320
2 Economy & Taxation -0.859
3 Education        -0.617
4 Immigration & Borders 0.637
5 National Security & Defense -0.137
6 <NA>            NA
```

7 Topic model 2: Structural Topic Model (time-aware, recommended)

LDA's post-hoc aggregation has no principled standard error on a trend. The **Structural Topic Model** lets topic prevalence depend on covariates; here, smoothly on year:

$$\theta_d \sim \text{LogisticNormal}(\Gamma^\top x_d, \Sigma), \quad x_d = [1, s(\text{year}_d)].$$

This is the cleaner tool for “how do topics evolve over time,” estimating the time effect jointly with the topics. STM extends beyond the SURV727 core toolkit.

```
processed <- textProcessor(
  documents = observed$text,
  metadata = observed |> dplyr::select(doc_id, year),
  lowercase = TRUE, removestopwords = TRUE, removenumbers = TRUE,
  removepunctuation = TRUE, stem = FALSE, verbose = FALSE
)
prepped <- prepDocuments(processed$documents, processed$vocab,
  processed$meta, lower.thresh = 5, verbose = FALSE)

stm_fit <- stm(
  documents = prepped$documents, vocab = prepped$vocab, K = dgp$K,
  prevalence = ~ s(year), data = prepped$meta,
  init.type = "Spectral", seed = 740, verbose = FALSE
)
```

```
stm_terms <- labelTopics(stm_fit, n = 8)
stm_top <- imap_dfr(seq_len(dgp$K),
  \(k, .) tibble(topic = k, term = stm_terms$frex[k, ]))
stm_topic_map <- align_topics(stm_top)
stm_topic_map
```

```
# A tibble: 6 x 2
  topic planted_name
<int> <chr>
1      1 Healthcare
2      2 Education
3      3 Immigration & Borders
4      4 Climate & Energy
5      5 Economy & Taxation
6      6 National Security & Defense
```

```
# Formal model-based test of the time effect (with uncertainty).
stm_effect <- estimateEffect(1:dgp$K ~ s(year), stm_fit,
  metadata = prepped$meta, uncertainty = "Global")
```

```
stm_gamma <- stm_fit$theta |>
  as_tibble(.name_repair = ~ as.character(seq_len(dgp$K))) |>
  mutate(doc_id = prepped$meta$doc_id, year = prepped$meta$year) |>
  pivot_longer(c(-doc_id, -year), names_to = "topic", values_to = "gamma") |>
```

```

mutate(topic = as.integer(topic)) |>
left_join(stm_topic_map, by = "topic")

stm_prev <- stm_gamma |>
group_by(year, planted_name) |>
summarise(est_prev = mean(gamma), .groups = "drop")

stm_prev |>
ggplot(aes(year, est_prev, color = planted_name)) +
geom_line(linewidth = 0.9) +
geom_line(data = truth_prevalence |> rename(planted_name = topic),
aes(year, true_prev), linetype = "dashed", linewidth = 0.5) +
scale_color_viridis_d(end = 0.92) +
scale_x_continuous(breaks = dgp$years) +
scale_y_continuous(labels = label_percent()) +
facet_wrap(~ planted_name) +
labs(x = NULL, y = "Share of attention",
title = "STM recovery vs. ground truth") +
theme(legend.position = "none",
axis.text.x = element_text(angle = 45, hjust = 1, size = 6))

```

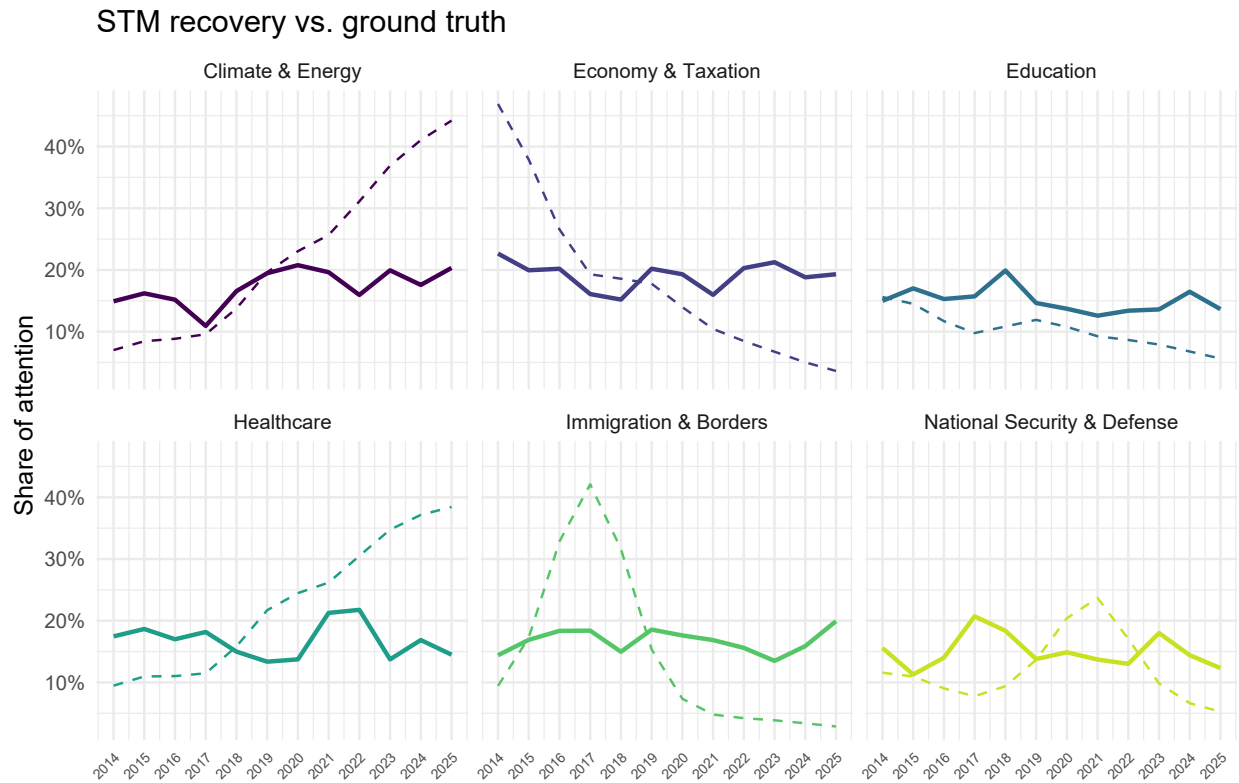


Figure 7: STM-recovered prevalence (theta averaged by year) vs. ground truth (dashed).

i Plotting the formal effect

`plot(stm_effect, covariate = "year", method = "continuous", topics = 1:6, model = stm_fit)` draws the modeled prevalence with confidence bands. The figure above averages `theta` by year so LDA, STM, and the weighted analysis (Section 10) share one comparable estimator; report the `estimateEffect` bands when you need inference on the trend itself.

Each document's dominant STM topic (used to condition sentiment in Section 8):

```
doc_dominant <- stm_gamma |>
  group_by(doc_id, year) |>
  slice_max(gamma, n = 1, with_ties = FALSE) |>
  ungroup() |>
  dplyr::select(doc_id, year, dominant_topic = planted_name)
```

8 Sentiment analysis (book Ch. 2 and Ch. 4)

Sentiment is treated, in TSE terms, as a **measurement** of latent tone. We start with the reliable, fully reproducible lexicon method, then quantify its known failure mode (negation), then (Section 9) cross-check against a local LLM.

The natural unit here is the **document (a paragraph)**, which the book notes works better than very large chunks. The Bing lexicon ships with `tidytext` and needs no license; AFINN/NRC require `textdata` plus a one-time acceptance, so AFINN-dependent chunks below are *gated* and skip gracefully if unavailable.

```
bing <- tryCatch(get_sentiments("bing"), error = function(e) NULL)
if (is.null(bing)) {
  # Fallback so the section always renders: a minimal lexicon from the valence
  # banks (on real data you will have the full Bing lexicon installed).
  bing <- bind_rows(
    tibble(word = valence_words$positive, sentiment = "positive"),
    tibble(word = valence_words$negative, sentiment = "negative")
  )
  message("Bing unavailable; using a minimal fallback lexicon (valence words
    ↪ only).")
}

# Per-document net sentiment (positive - negative word counts), Bing.
doc_sent_bing <- tokens |>
  inner_join(bing, by = "word") |>
  count(doc_id, year, sentiment) |>
  pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) |>
  mutate(net_bing = positive - negative)
```

```
# Re-attach documents with zero sentiment words (so means are not biased).
doc_sent_bing <- observed |>
  dplyr::select(doc_id, year) |>
  left_join(doc_sent_bing, by = c("doc_id", "year")) |>
  mutate(across(c(positive, negative, net_bing), \(x) coalesce(x, 0L)))
```

```
doc_sent_bing |>
  group_by(year) |>
  summarise(mean_net = mean(net_bing), .groups = "drop") |>
  ggplot(aes(year, mean_net)) +
  geom_line(linewidth = 0.9, color = viridisLite::viridis(1, begin = 0.3)) +
  geom_point(size = 1.4) +
  geom_hline(yintercept = 0, linetype = "dotted", linewidth = 0.3) +
  scale_x_continuous(breaks = dgp$years) +
  labs(x = NULL, y = "Mean net sentiment (Bing)",
       title = "Overall sentiment over time") +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))
```

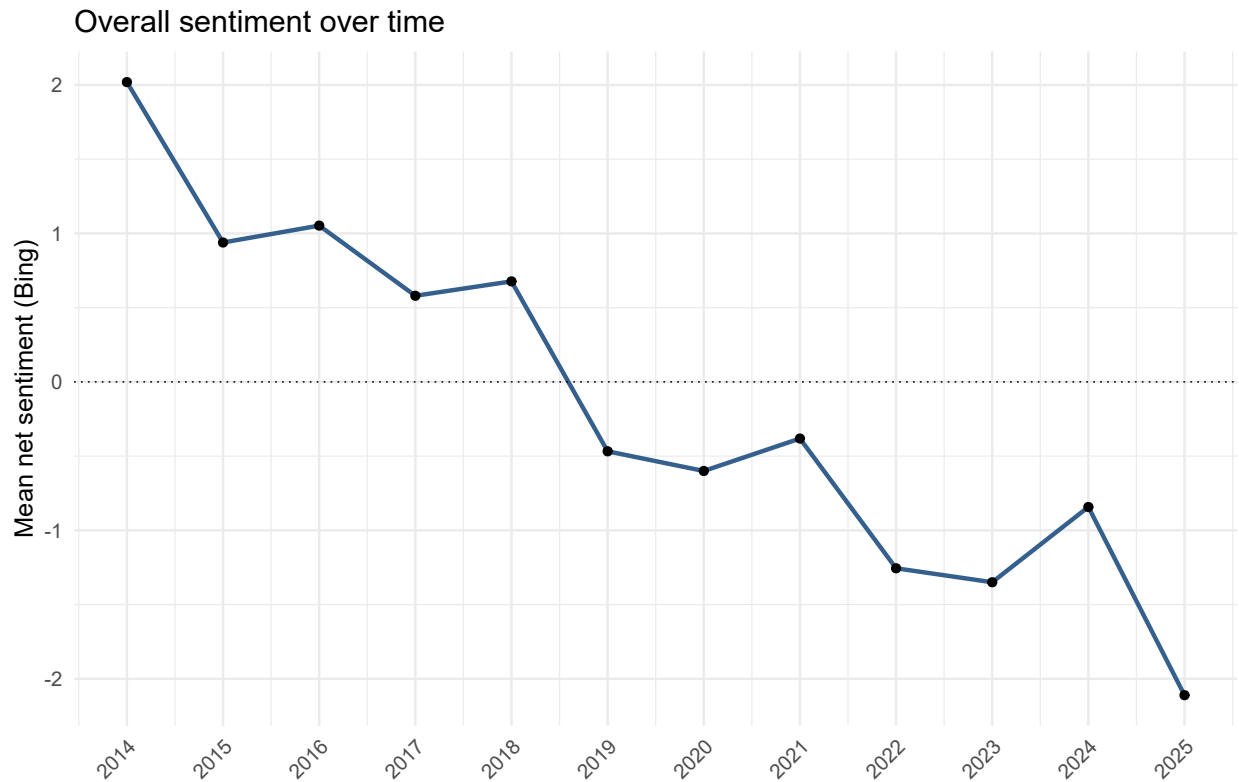


Figure 8: Mean per-document Bing net sentiment by year (the planted souring trend).

8.1 Sentiment by topic over time, validated against truth

Joining each document's net sentiment to its dominant STM topic gives sentiment trajectories per policy area, which we validate against the planted net valence.

```
sent_topic <- doc_sent_bing |>
  left_join(doc_dominant, by = c("doc_id", "year")) |>
  filter(!is.na(dominant_topic)) |>
  group_by(year, dominant_topic) |>
  summarise(mean_net = mean(net_bing), .groups = "drop")

# Put estimate and truth on a common (z-scored) scale for shape comparison.
z <- function(x) (x - mean(x)) / sd(x)
truth_z <- valence_truth |> group_by(topic) |> mutate(net_z = z(net)) |>
  ungroup()
est_z <- sent_topic |> group_by(dominant_topic) |> mutate(net_z = z(mean_net))
  ungroup()

ggplot(est_z, aes(year, net_z, color = dominant_topic)) +
  geom_line(linewidth = 0.9) +
  geom_line(data = truth_z |> rename(dominant_topic = topic),
            aes(year, net_z), linetype = "dashed", linewidth = 0.5) +
  scale_color_viridis_d(end = 0.92) +
  scale_x_continuous(breaks = dgp$years) +
  facet_wrap(~ dominant_topic) +
  labs(x = NULL, y = "Net sentiment (z-scored)",
       title = "Sentiment by topic: estimate vs. planted truth") +
  theme(legend.position = "none",
        axis.text.x = element_text(angle = 45, hjust = 1, size = 6))
```

Sentiment by topic: estimate vs. planted truth

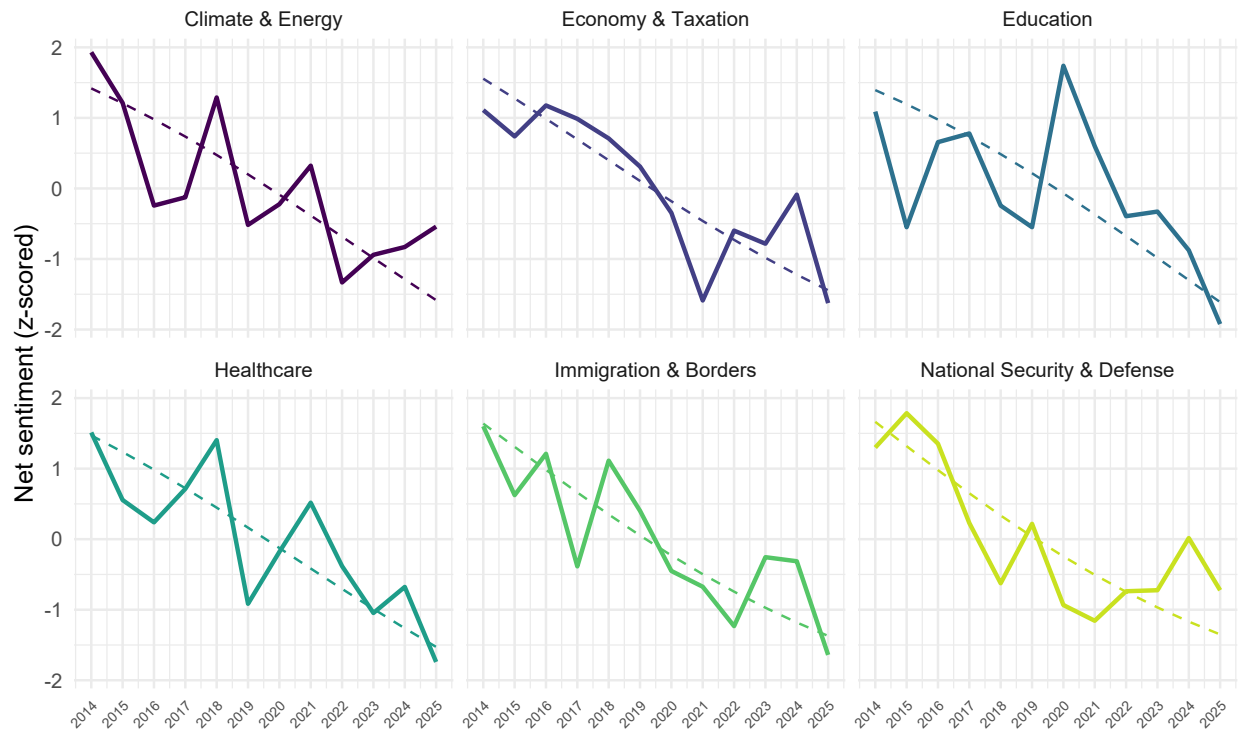


Figure 9: Estimated sentiment by topic over time (lines) and standardized planted valence (dashed).

```
sent_topic |>
  left_join(valence_truth, by = c("year", "dominant_topic" = "topic")) |>
  group_by(dominant_topic) |>
  summarise(correlation = cor(mean_net, net), .groups = "drop")
```

```
# A tibble: 6 x 2
  dominant_topic correlation
  <chr>          <dbl>
1 Climate & Energy 0.731
2 Economy & Taxation 0.845
3 Education        0.575
4 Healthcare       0.797
5 Immigration & Borders 0.819
6 National Security & Defense 0.801
```

8.2 Negation diagnostic (book Ch. 4): how much does unigram sentiment misread?

Unigram lexicons score “good” and “support” as positive even in “not good” or “never support.” Tokenizing into bigrams and finding sentiment words preceded by a negator quantifies the exposure. This uses AFINN (numeric), so it is gated.

```

negators <- c("not", "no", "never", "without")

negated <- observed |>
  unnest_tokens(bigram, text, token = "ngrams", n = 2) |>
  filter(!is.na(bigram)) |>
  separate(bigram, c("word1", "word2"), sep = " ") |>
  filter(word1 %in% negators) |>
  inner_join(afinn, by = c("word2" = "word")) |>
  count(word1, word2, value, sort = TRUE) |>
  mutate(contribution = n * value)

if (nrow(negated) == 0) {
  message("No negated sentiment words found (expected: this synthetic corpus
    ↪ contains little negation by construction).")
} else {
  negated |>
    slice_max(abs(contribution), n = 20) |>
    mutate(word2 = reorder(word2, contribution)) |>
    ggplot(aes(contribution, word2, fill = contribution > 0)) +
    geom_col(show.legend = FALSE) +
    scale_fill_viridis_d(end = 0.7) +
    labs(x = "AFINN value x occurrences", y = "Word preceded by a negator",
         title = "Sentiment words that may be misread due to negation")
}

```

AFINN not available. Install `textdata` and run `textdata::lexicon_afinn()` once interactively to accept the license, then re-render to enable the negation diagnostic.

Lexicon caveats (stated by the book)

Lexicon sentiment is **unigram-based** and ignores qualifiers (“no good”, “not true”); it was **validated on specific text types** (reviews, tweets), so applying it to political prose risks domain mismatch; and **chunk size matters** (sentence- or paragraph-sized text works better than many-paragraph blocks). Real political commentary is dense with negation, so run the diagnostic above and consider a negation-corrected score (flip the sign of negated words) or a context-aware tool (`sentimentr`) or the LLM route below. On this synthetic corpus negation is rare by construction, so the diagnostic is shown mainly as the procedure to run on real data.

9 Optional: AI-assisted module with a local model (Gemma via Ollama)

You can run a local LLM through `ellmer`’s Ollama backend, so no data leaves your machine and there is no API cost. Two auditable uses follow: topic labeling and sentiment, each cross-checked. All AI chunks are `eval: false` so the PDF renders without Ollama running; flip to `eval: true`

once `ollama serve` is up and the model is pulled. Swapping to a hosted model is a one-line change (`chat_anthropic()`, `chat_openai()`, ...).

```
library(ellmer)

# Confirm the exact local tag, then set it here (e.g. "gemma3", "gemma4",
#   ↪ "llama4").
# ellmer::models_ollama()
ollama_model <- "gemma3"

# temperature 0 + fixed seed for reproducibility; local => private, free.
make_chat <- function(system_prompt) {
  chat_ollama(
    system_prompt = system_prompt,
    model = ollama_model,
    params = params(temperature = 0, seed = 1)
  )
}
```

9.1 Use 1 - LLM topic labeling

Turning each topic's top terms into a human-readable label is low risk: the model sees only anonymized term lists, and a human reviews the output.

```
labeler <- make_chat(paste(
  "You label political topic models. Given the top terms of one topic, return a",
  "concise 2-4 word policy label and a one-sentence description, based only on",
  "the terms provided."
))

label_type <- type_object(
  label = type_string("A concise 2-4 word policy-area label."),
  description = type_string("One sentence describing the topic.")
)

ai_labels <- stm_top |>
  summarise(terms = paste(term, collapse = ", "), .by = topic) |>
  mutate(result = map(terms, \(tt)
    labeler$chat_structured(glue("Top terms: {tt}"), type = label_type))) |>
  unnest_wider(result)
ai_labels
```

9.2 Use 2 - LLM sentiment, cross-checked against the lexicon

Classify each post's sentiment with the local model, then compare to the Bing score: Spearman correlation on the continuous signal and Cohen's kappa on the sign. Agreement is convergent-

validity evidence; disagreement flags uncertain measurements. **Process one post per call** — Ollama’s default input cap is 2048 tokens, which a paragraph fits easily but a batch would not.

```
rater <- make_chat(paste(
  "You rate the sentiment of a political post toward the policy it discusses.",
  "Return a label (negative/neutral/positive) and a score from -1 (very",
  "negative) to 1 (very positive).")
))

sent_type <- type_object(
  label = type_enum(c("negative", "neutral", "positive"), "Overall sentiment."),
  score = type_number("Sentiment score from -1 to 1.")
)

set.seed(745)
ai_check <- observed |> slice_sample(n = 200)

# Small local models can occasionally return non-conforming output; guard each
# call.
safe_rate <- function(txt) {
  tryCatch(rater$chat_structured(txt, type = sent_type),
    error = function(e) list(label = NA_character_, score = NA_real_)
  )
}

ai_sent <- ai_check |>
  mutate(res = map(text, safe_rate)) |>
  unnest_wider(res) |>
  left_join(doc_sent_bing, by = c("doc_id", "year"))

# Convergent validity
spearman <- cor(ai_sent$score, ai_sent$net_bing,
  method = "spearman", use = "complete.obs")
kappa <- irr::kappa2(
  ai_sent |>
    transmute(ai = label,
      lex = case_when(net_bing > 0 ~ "positive",
        net_bing < 0 ~ "negative",
        TRUE ~ "neutral")) |>
    drop_na()
)
list(spearman = spearman, kappa = kappa)
```

i Convenience alternative: the {mall} package

mall wraps **ellmer** for column-wise LLM operations, e.g. `mall::llm_sentiment()` over a text column after wiring a backend with `mall::llm_use()` (check current **mall** docs for the exact Ollama wiring). It is the fastest path to “AI sentiment through an R package,” but the explicit

ellmer route above gives more control and is what is specified here.

⚠ AI validity and reproducibility caveats

LLM labels and scores are **measurements with error** in the TSE “measurement” cell. Even at temperature 0, local models can vary across versions and quantizations and are sensitive to prompt wording. Pin the model tag, log prompts and raw responses, validate against the lexicon (above), and never treat an LLM label as ground truth. Small local models also produce less reliable structured output than hosted ones — hence the per-call guard.

10 Inference under sampling: correcting selection bias

Sections 7-8 estimated prevalence and sentiment *in the observed sample*, which over-represents recent and non-controversial posts. This section makes the correction explicit with design-based estimation — the survey-methods payoff.

10.1 The estimator

With unequal inclusion probabilities π_d , a population mean (of topic share or of sentiment) in year t is the Horvitz-Thompson / weighted mean

$$\hat{y}_t = \frac{\sum_{d \in s_t} w_d y_d}{\sum_{d \in s_t} w_d}, \quad w_d = \frac{1}{\pi_d},$$

and the variance penalty from unequal weighting is the Kish design effect

$$\text{deff}_{\text{Kish}} = 1 + \text{CV}^2(w) = \frac{n \sum_d w_d^2}{(\sum_d w_d)^2}.$$

```
# Analyst-estimable propensity: predict capture from year only (MAR assumption).
prop_model <- glm(observed ~ year_c, family = binomial, data = corpus_pop)
corpus_pop <- corpus_pop |>
  mutate(pi_hat_mar = predict(prop_model, type = "response"))

# Document-level analysis frame: STM topic shares (wide) + Bing sentiment +
  ↳ weights.
topic_cols <- stm_topic_map$planted_name

stm_wide <- stm_fit$theta |>
  as_tibble(.name_repair = ~ stm_topic_map$planted_name[
    match(seq_len(dgp$K), stm_topic_map$topic)]) |>
  mutate(doc_id = prepped$meta$doc_id)
```

```

analysis <- stm_wide |>
  left_join(dplyr::select(doc_sent_bing, doc_id, net_bing), by = "doc_id") |>
  left_join(dplyr::select(corpus_pop, doc_id, year, pi_true, pi_hat_mar),
            by = "doc_id") |>
  mutate(w_ipw = 1 / pi_true, w_mar = 1 / pi_hat_mar)

kish_deff <- with(analysis, length(w_ipw) * sum(w_ipw^2) / sum(w_ipw)^2)
glue("Kish design effect (correct weights): {round(kish_deff, 2)} ",
     "(effective n approx {round(nrow(analysis) / kish_deff)}).")

```

Kish design effect (correct weights): 1.12 (effective n approx 812).

10.2 Prevalence: naive vs. corrected

```

naive_prev <- analysis |>
  group_by(year) |>
  summarise(across(all_of(topic_cols), mean), .groups = "drop") |>
  pivot_longer(-year, names_to = "topic", values_to = "prev") |>
  mutate(estimator = "Naive (unweighted)")

ipw_prev <- analysis |>
  as_survey_design(ids = 1, weights = w_ipw) |>
  group_by(year) |>
  summarise(across(all_of(topic_cols), survey_mean)) |>
  dplyr::select(year, all_of(topic_cols)) |>
  pivot_longer(-year, names_to = "topic", values_to = "prev") |>
  mutate(estimator = "IPW (correct weights)")

truth_long <- truth_prevalence |> rename(prev = true_prev) |>
  mutate(estimator = "Population truth")

bind_rows(truth_long, naive_prev, ipw_prev) |>
  mutate(topic = factor(topic, levels = topic_names)) |>
  ggplot(aes(year, prev, color = estimator, linetype = estimator)) +
  geom_line(linewidth = 0.8) +
  scale_color_viridis_d(end = 0.85) +
  scale_x_continuous(breaks = dgp$years) +
  scale_y_continuous(labels = label_percent()) +
  facet_wrap(~ topic) +
  labs(x = NULL, y = "Share of attention", color = NULL, linetype = NULL,
       title = "Selection bias and its correction: prevalence") +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 45, hjust = 1, size = 6))

```

Selection bias and its correction: prevalence

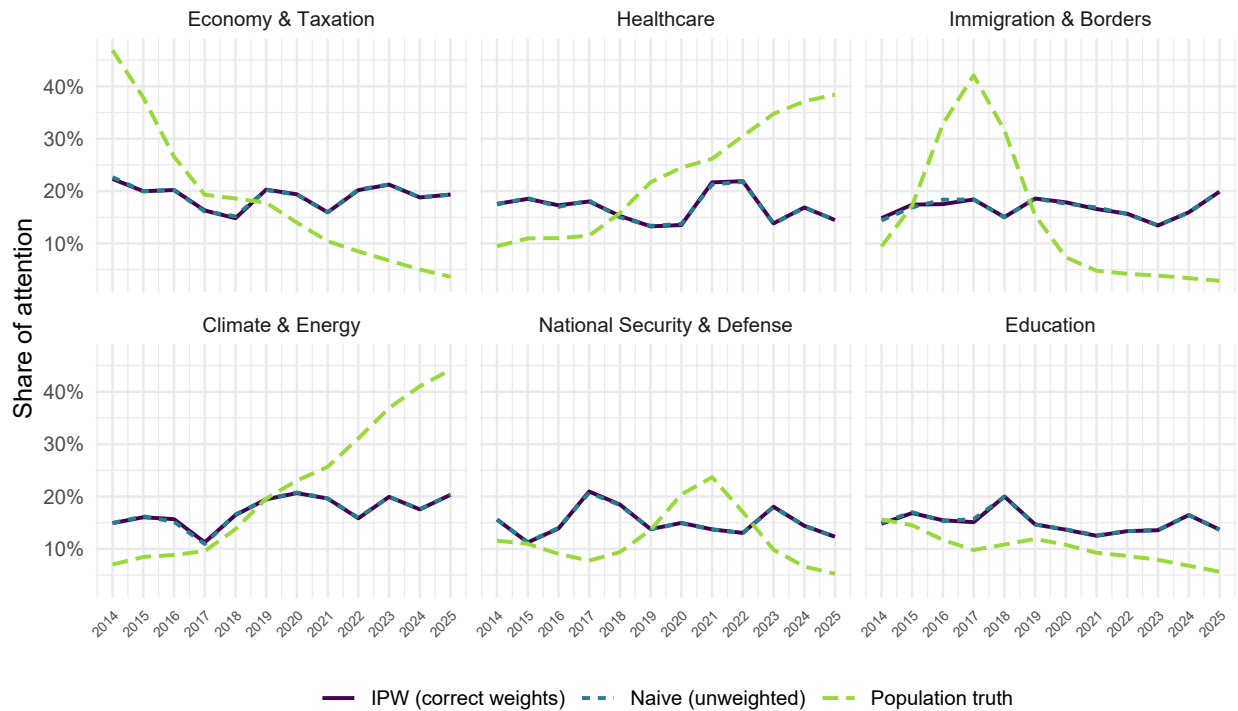


Figure 10: Naive sample means are biased; IPW with correct inclusion probabilities recovers population prevalence.

10.3 Sentiment: naive vs. corrected

```
naive_sent <- analysis |>
  group_by(year) |>
  summarise(mean_net = mean(net_bing), .groups = "drop") |>
  mutate(estimator = "Naive (unweighted)")
```

```
ipw_sent <- analysis |>
  as_survey_design(ids = 1, weights = w_ipw) |>
  group_by(year) |>
  summarise(mean_net = survey_mean(net_bing)) |>
  dplyr::select(year, mean_net) |>
  mutate(estimator = "IPW (correct weights)")
```

```
# Recompute Bing net sentiment on the FULL population (truth reference only).
pop_tokens <- corpus_pop |>
  dplyr::select(doc_id, year, text) |>
  unnest_tokens(word, text) |>
  anti_join(get_stopwords(), by = "word") |>
```

```

filter(!str_detect(word, "\\d"))

pop_doc_sent <- corpus_pop |>
  dplyr::select(doc_id, year) |>
  left_join(
    pop_tokens |>
      inner_join(bing, by = "word") |>
      count(doc_id, sentiment) |>
      pivot_wider(names_from = sentiment, values_from = n, values_fill = 0) |>
      mutate(net_bing = positive - negative) |>
      dplyr::select(doc_id, net_bing),
    by = "doc_id"
  ) |>
  mutate(net_bing = coalesce(net_bing, 0L))

truth_sent_line <- pop_doc_sent |>
  group_by(year) |>
  summarise(mean_net = mean(net_bing), .groups = "drop") |>
  mutate(estimator = "Population truth")

bind_rows(truth_sent_line, naive_sent, ipw_sent) |>
  ggplot(aes(year, mean_net, color = estimator, linetype = estimator)) +
  geom_line(linewidth = 0.9) +
  geom_hline(yintercept = 0, linetype = "dotted", linewidth = 0.3) +
  scale_color_viridis_d(end = 0.85) +
  scale_x_continuous(breaks = dgp$years) +
  labs(x = NULL, y = "Mean net sentiment", color = NULL, linetype = NULL,
       title = "Selection bias and its correction: sentiment") +
  theme(legend.position = "bottom",
        axis.text.x = element_text(angle = 45, hjust = 1))

```

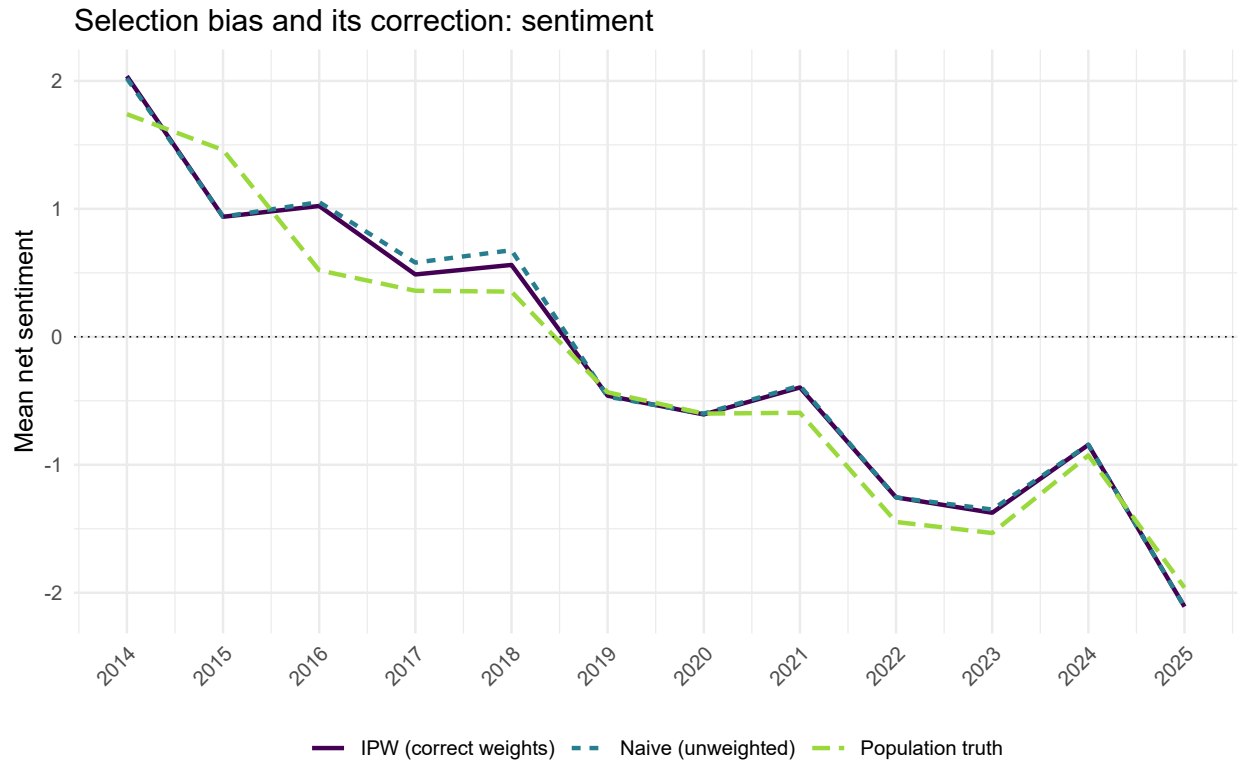


Figure 11: Sentiment correction with a population-truth reference.

10.4 Sensitivity analysis: how much does the assumption matter?

The MNAR mechanism deletes controversial posts based on *content*, which an analyst cannot observe directly. MAR-only weights (year alone) therefore cannot fully correct the bias. The residual gap is the honest measure of how far conclusions depend on an unverifiable assumption.

```
mar_prev <- analysis |>
  as_survey_design(ids = 1, weights = w_mar) |>
  group_by(year) |>
  summarise(across(all_of(topic_cols), survey_mean)) |>
  dplyr::select(year, all_of(topic_cols)) |>
  pivot_longer(-year, names_to = "topic", values_to = "prev") |>
  mutate(estimator = "IPW (MAR / year only)")

bind_rows(truth_long,
  mar_prev,
  ipw_prev |> mutate(estimator = "IPW (correct)")) |>
  filter(topic %in% topic_names[idx_control]) |>
  mutate(topic = factor(topic, levels = topic_names)) |>
  ggplot(aes(year, prev, color = estimator, linetype = estimator)) +
  geom_line(linewidth = 0.9) +
  scale_color_viridis_d(end = 0.85) +
```

```

scale_x_continuous(breaks = dgp$years) +
scale_y_continuous(labels = label_percent()) +
facet_wrap(~ topic) +
labs(x = NULL, y = "Share of attention", color = NULL, linetype = NULL,
      title = "Sensitivity to the missingness assumption (controversial
        ↪ topics)") +
theme(legend.position = "bottom",
      axis.text.x = element_text(angle = 45, hjust = 1, size = 6))

```

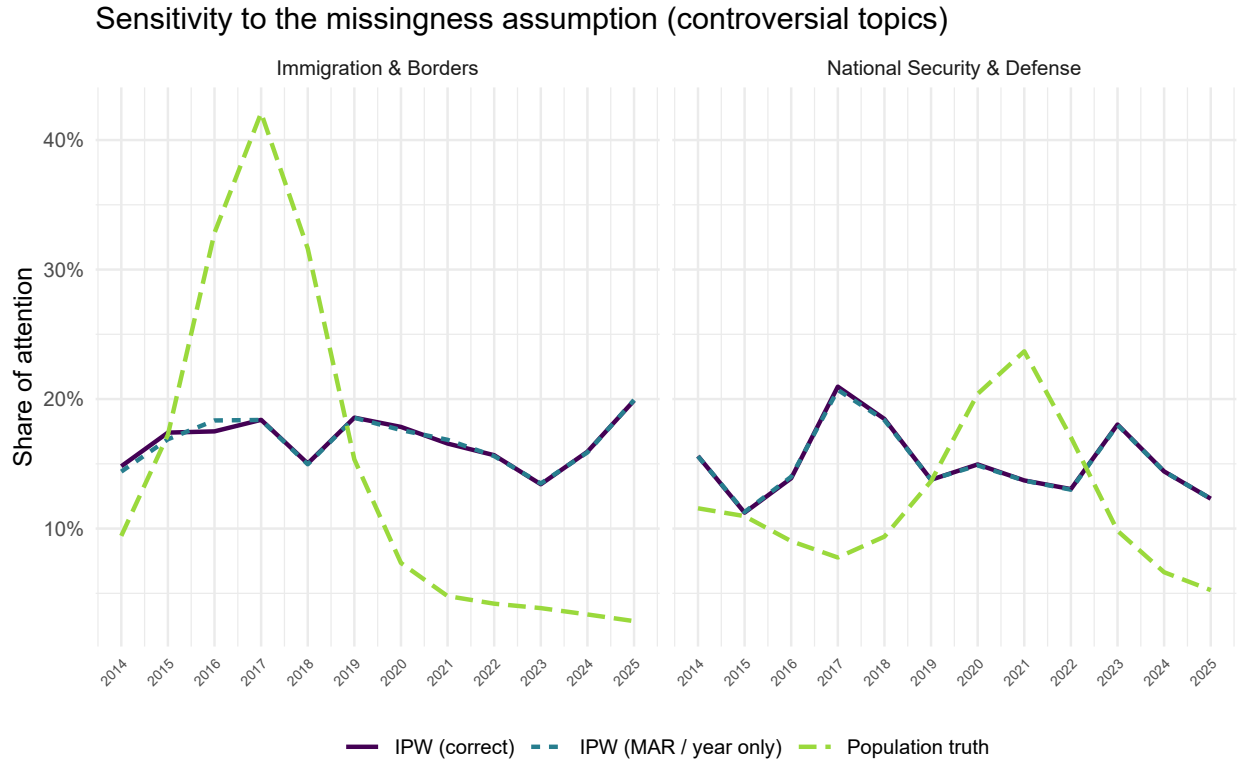


Figure 12: MAR-only weighting (year alone) leaves residual bias for content-dependent topics.

11 Adapting this workflow to the real corpus

1. **Replace Section 4** with a read of your scraped corpus: a tibble of `doc_id`, `date`, `text`; derive `year`. Delete every `theta`, `dominant`, `pi_true`, `observed`, and `valence_*` column — those exist only because truth was known.
2. **Confront coverage directly.** You cannot observe π_d . The principled estimate of *how many posts are missing* is capture-recapture (dual-system estimation): collect from two independent sources (live scrape + archive) and estimate the unobserved total from the overlap. Document the frame and its gaps.
3. **Estimate inclusion propensities** from observable predictors (post age, source, length, engagement); this yields the MAR estimator, not the correct one. State the residual-MNAR

risk; the sensitivity analysis is your honest uncertainty band.

4. **Choose K** with the documented perplexity / **searchK** procedure plus interpretability.
5. **For sentiment**, run the negation diagnostic, prefer paragraph-sized units, consider a context-aware method, and validate any LLM scores against a lexicon baseline as in Section 9.
6. **Keep the validation mindset.** Without ground truth, validation shifts to topic stability across seeds, held-out coherence, human coding of a sample, and lexicon-vs-LLM agreement.

12 Limitations and threats to validity

- **Frame / coverage.** Capturable posts are unknown; deletions and platform gaps are not random. Capture-recapture bounds this; nothing eliminates it.
- **MNAR selection.** If capture depends on content, no weighting on observed covariates fully corrects the bias. Conclusions are conditional on the missingness model; report the sensitivity range.
- **Measurement: topic instability.** Topics shift with K , seed, preprocessing, and trimming. Report robustness.
- **Measurement: temporal non-invariance.** LDA and STM assume a *time-invariant* topic-word distribution β ; over a decade, framing drifts, so a fixed β can mis-measure later years. Dynamic Topic Models (Blei & Lafferty 2006) relax this; treat decade-long trends as provisional until checked.
- **Measurement: sentiment.** Unigram lexicons miss negation and domain shift; LLM scores are non-deterministic and version-dependent. Triangulate.
- **Construct validity.** “Topics” are word-co-occurrence patterns and “sentiment” is lexical tone, neither of which is the politician’s intent; a rising topic share is attention, not endorsement.
- **Simulation gap.** Real topics overlap and real negation is common, so real-world recovery will be noisier than shown here.

13 References

! Verify before circulating

Per standing practice, confirm every citation against its primary source. The list below is from memory and is **not** independently verified in this render.

- Silge, J., & Robinson, D. (2017). *Text Mining with R: A Tidy Approach*. O’Reilly. <https://www.tidytextmining.com/>
- Blei, D. M., Ng, A. Y., & Jordan, M. I. (2003). Latent Dirichlet Allocation. *Journal of Machine Learning Research*, 3, 993-1022.
- Blei, D. M., & Lafferty, J. D. (2006). Dynamic Topic Models. *ICML*.
- Roberts, M. E., Stewart, B. M., & Tingley, D. (2019). stm: An R Package for Structural Topic Models. *Journal of Statistical Software*, 91(2).
- Roberts, M. E., et al. (2014). Structural Topic Models for Open-Ended Survey Responses. *American Journal of Political Science*, 58(4).
- Grimmer, J., & Stewart, B. M. (2013). Text as Data. *Political Analysis*, 21(3).

- Grimmer, J., Roberts, M. E., & Stewart, B. M. (2022). *Text as Data*. Princeton.
- Salganik, M. J. (2018). *Bit by Bit: Social Research in the Digital Age*. Princeton University Press.
- Japac, L., et al. (2015). Big Data in Survey Research: AAPOR Task Force Report. *Public Opinion Quarterly*, 79(4).
- Sen, I., et al. (2021). A Total Error Framework for Digital Traces of Human Behavior on Online Platforms. *Public Opinion Quarterly*, 85(S1). (*verify*)
- Gilardi, F., Alizadeh, M., & Kubli, M. (2023). ChatGPT Outperforms Crowd Workers for Text-Annotation Tasks. *PNAS*, 120(30).
- Kish, L. (1965). *Survey Sampling*. Wiley.
- Lohr, S. L. (2022). *Sampling: Design and Analysis* (3rd ed.). CRC Press.
- Heeringa, S. G., West, B. T., & Berglund, P. A. (2017). *Applied Survey Data Analysis* (2nd ed.). CRC Press.